

# Deep Generative Models

## Chapter 4: Generative Adversarial Networks

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering  
University of Toronto

Summer 2026

# Questioning Validity of GAN

- + Though *intuitive*, is there any ways to show that this training approach is indeed leading to *sampling from data distribution*?
- Well! We can claim this, if we can show the *following*

Let  $G_{\mathbf{w}^*}$  and  $D_{\phi^*}$  be *solutions* to the *min-max game*; then,  $x = G_{\mathbf{w}^*}(z)$  for  $z \sim Q(z)$  is *approximately* distributed by *data distribution*

To show this arguments let's consider the following *idealistic* assumptions

- *Discriminator*  $D_{\phi}$  can realize any function  $D$  exactly
- *Generator*  $G_{\mathbf{w}}$  can realize any mapping  $G$  exactly
  - ↳ Discriminator and generator are *infinitely-deep NNs*
- Dataset  $\mathbb{D}$  has *infinitely large number of samples*
  - ↳ Our estimated average is *accurate*, i.e.,  $\hat{\mathbb{E}} \rightsquigarrow \mathbb{E}$

## Min-Max Game in *Idealistic Setting*

With an **ideal** setting: the objective of the **min-max problem** becomes

$$\mathcal{L}(G, D) = \mathbb{E}_{x \sim P} \{\log D(x)\} + \mathbb{E}_{z \sim Q} \{\log (1 - D(G(z)))\}$$

Since we assume ***G* is any generator**: we can say

$x = G(z)$  can be distributed by **any  $\hat{P}$**  on  $\mathbb{X}$

↳ By **changing  $G \rightsquigarrow \hat{P}$**  is changed<sup>a</sup>

---

<sup>a</sup>Recall that  **$\hat{P}$**  is **not** necessarily computable!

With these definitions, we could say

$$\begin{aligned} \mathcal{L}(G, D) &= \mathbb{E}_{x \sim P} \{\log D(x)\} + \mathbb{E}_{x \sim \hat{P}} \{\log (1 - D(x))\} \\ &\equiv \mathcal{L}(\hat{P}, D) \end{aligned}$$

## Optimal Players: *Discriminator*

Let's start with finding theoretically-*optimal* generator and discriminator: we could expand the objective as

$$\begin{aligned}\mathcal{L}(\hat{P}, D) &= \mathbb{E}_{x \sim P} \{\log D(x)\} + \mathbb{E}_{x \sim \hat{P}} \{\log(1 - D(x))\} \\ &= \int_{\mathbb{X}} \log D(x) P(x) dx + \int_{\mathbb{X}} \log(1 - D(x)) \hat{P}(x) dx \\ &= \int_{\mathbb{X}} \log D(x) P(x) + \log(1 - D(x)) \hat{P}(x) dx\end{aligned}$$

The *min-max game* first *maximizes*  $\mathcal{L}$  over  $D$ : we can find  $D^*$  by setting

$$\frac{d}{dD} \mathcal{L}(\hat{P}, D) \stackrel{!}{=} 0$$

- + How can we compute derivative with respect to a function?!
- We should use *Radon-Nikodym derivative*, but let us do it *naively*!

# Optimal Discriminator: Naive Analysis

## ! Attention

*Following analysis is simplified and not accurate, but reflects key idea*

*Under some regularity conditions, we can find the optimal  $D$  by writing*

$$\begin{aligned}\frac{d}{dD} \mathcal{L}(\hat{P}, D) &= \frac{d}{dD} \int_{\mathcal{X}} \log(D(x)) P(x) + \log(1 - D(x)) \hat{P}(x) dx \\ &= \int_{\mathcal{X}} \frac{d}{dD(x)} \left[ \log(D(x)) P(x) + \log(1 - D(x)) \hat{P}(x) \right] dx\end{aligned}$$

*and setting the integrand to zero at every  $x$*

$$\begin{aligned}\frac{d}{dD(x)} \left[ \log(D(x)) P(x) + \log(1 - D(x)) \hat{P}(x) \right] &= 0 \\ \frac{P(x)}{D(x)} - \frac{\hat{P}(x)}{1 - D(x)} &= 0\end{aligned}$$

## Optimal Discriminator: Naive Analysis

This conclude that

$$D^*(x) = \frac{P(x)}{P(x) + \hat{P}(x)}$$

is the **optimal discriminator** that maximizes the objective

### Attention

Note that  $D^*(x)$  is not computable!

↳ We have **no access** to the **data distribution**

↳ We **cannot** compute explicitly  $\hat{P}(x)$

But, if the discriminator model is **deep enough** and we have a **large number of samples**  $\rightsquigarrow$  we can approximate it by a **computational model**  $D_\phi$

# Optimal Generator

To find **optimal** generator: we can use  $D^*$  and solve *outer minimization*, i.e.,

$$\min_{\hat{P}} \max_D \mathcal{L}(\hat{P}, D) = \min_{\hat{P}} \mathcal{L}(\hat{P}, D^*)$$

This means that the **optimal generator distribution**  $\hat{P}$  is a solution to

$$\min_{\hat{P}} \mathcal{L}(\hat{P}, D^*) = \min_{\hat{P}} \mathbb{E}_{x \sim P} \{\log D^*(x)\} + \mathbb{E}_{x \sim \hat{P}} \{\log (1 - D^*(x))\}$$

This reduces to

$$\min_{\hat{P}} \mathbb{E}_{x \sim P} \left\{ \log \frac{P(x)}{P(x) + \hat{P}(x)} \right\} + \mathbb{E}_{x \sim \hat{P}} \left\{ \log \left( 1 - \frac{P(x)}{P(x) + \hat{P}(x)} \right) \right\}$$

# Optimal Generator: Divergence Minimizer

## Average Distribution

Let us define the *average* of model and data distribution as

$$A_{P,\hat{P}}(x) = 0.5 \left( P(x) + \hat{P}(x) \right)$$

which is *still a distribution* defined on  $\mathbb{X}$

With this definition: we could write the optimization as

$$\begin{aligned} & \min_{\hat{P}} \mathbb{E}_{x \sim P} \left\{ \log \frac{P(x)}{2A_{P,\hat{P}}(x)} \right\} + \mathbb{E}_{x \sim \hat{P}} \left\{ \log \frac{\hat{P}(x)}{2A_{P,\hat{P}}(x)} \right\} \\ &= \min_{\hat{P}} \mathbb{E}_{x \sim P} \left\{ \log \frac{P(x)}{A_{P,\hat{P}}(x)} \right\} + \mathbb{E}_{x \sim \hat{P}} \left\{ \log \frac{\hat{P}(x)}{A_{P,\hat{P}}(x)} \right\} - 2 \log 2 \end{aligned}$$



# Optimal Generator: *Divergence Minimizer*

## Recall: KL Divergence

The KL divergence between  $P$  and  $Q$  is defined as

$$D_{\text{KL}}(P\|Q) = \mathbb{E}_{x \sim P} \left\{ \log \frac{P(x)}{Q(x)} \right\}$$

We can thus say that

$$\begin{aligned} & \min_{\hat{P}} \mathbb{E}_{x \sim P} \left\{ \log \frac{P(x)}{A_{P, \hat{P}}(x)} \right\} + \mathbb{E}_{x \sim \hat{P}} \left\{ \log \frac{\hat{P}(x)}{A_{P, \hat{P}}(x)} \right\} \\ &= \min_{\hat{P}} \boxed{D_{\text{KL}}(P\|A_{P, \hat{P}}) + D_{\text{KL}}(\hat{P}\|A_{P, \hat{P}})} \end{aligned}$$

divergence metric

# Jensen-Shannon Divergence

## Jensen-Shannon Divergence

The Jensen-Shannon divergence between  $P$  and  $\hat{P}$  is defined as

$$D_{\text{JS}} \left( P \parallel \hat{P} \right) = D_{\text{KL}} \left( P \parallel A_{P, \hat{P}} \right) + D_{\text{KL}} \left( \hat{P} \parallel A_{P, \hat{P}} \right)$$

In Assignment 2 we played around with *JS divergence* to see the following

## KL Divergence vs JS Divergence

Under some regularity conditions on  $P$  and  $\hat{P}$ , we can say that

$$D_{\text{JS}} \left( P \parallel \hat{P} \right) = 0 \iff D_{\text{KL}} \left( P \parallel \hat{P} \right) = 0 \iff P = \hat{P}$$

## Optimal Generator: Divergence Minimizer

In GAN training, we find a *generator distribution*  $\hat{P}$  is

$$\hat{P}^{\star} = \operatorname{argmin}_{\hat{P}} D_{\text{JS}} \left( P \parallel \hat{P} \right) \longleftrightarrow \hat{P}^{\star} = \operatorname{argmin}_{\hat{P}} D_{\text{KL}} \left( P \parallel \hat{P} \right)$$

In other words, as we train using *min-max game*

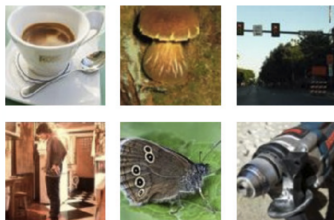
distribution of *generated sample*  $x = G(z)$  tends to *data distribution*

### Moral of Story: Implicit MLE via GAN

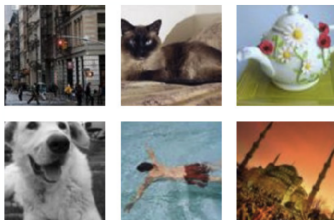
Using *min-max training* strategy of GAN we *implicitly* train a generator with *minimal KL divergence*  $\equiv$  *maximal likelihood* at its output

# Sample Outputs of GAN

*Trained on the dataset*



*GAN is able to sample*



# Vanilla GAN: Stability Challenge

Vanilla GAN suffers from various **practical issues**

- Training loop is **unstable**
  - ↳ Since we use **gradient descent** for **min-max** it could easily diverge
- **Mode collapse** happens frequently
  - ↳ **Generator** gives **a few** good samples all the time to fool discriminator
  - ↳ This results in **lack** of **diversity**
- Computational vanilla GANs are **vulnerable** to **vanishing gradient**
  - ↳ Objective function returns small gradients

To overcome these issues, **Wasserstein GAN** was proposed